

DESARROLLO ORIENTADO A OBJETOS [DOPO]

2026-2

Ciclo 2

Materia: Desarrollo orientado a objetos

Grupo: 7

Integrantes

Jhon Eduard Duran Ceballos

ID: 1000100325

Correo: jhon.duran-c@mail.escuelaing.edu.co

Andres Steven Oyuela Mendez

ID: 1000109048

Correo: andres.oyuela-m@mail.escuelaing.edu.co

Introducción

El objetivo es diseñar e implementar un simulador de máquina tragamonedas que aplique rigurosamente los principios de POO: encapsulamiento, cohesión, bajo acoplamiento y extensibilidad. Utilizando como base el framework de clases shapes, el sistema se construye de manera incremental a través de 5 mini-ciclos de desarrollo. Este enfoque metodológico asegura que cada componente sea validado, documentado y probado, culminando en una entrega que cumple con estándares de diseño en Astah, control de versiones en Git y una retrospectiva técnica detallada

Métodos de Uso

1. **swap(int wheel1, int wheel2):** Ejecuta la permuta de dos elementos dentro del arreglo de ruedas, aplicando un control de excepciones para asegurar que los parámetros se mantengan en los umbrales operativos.
2. **lock(int wheel):** Modifica el estado interno a inmovilizado y actualiza la representación gráfica con un borde carmesí, garantizando que el componente permanezca estable ante eventos de rotación global.
3. **unlock(int wheel):** Revierte el estado de fijación del objeto, restableciendo la estética original del marco y habilitando nuevamente la transición de símbolos durante la ejecución del giro.
4. **spin(int wheel, int steps):** Desplaza un disco específico una magnitud definida de posiciones; integra una secuencia de refresco visual en el Canvas mediante retardos programados si la visibilidad está activa.
5. **spin(String[] setSymbols):** Establece de forma imperativa los valores del sistema mediante un vector de cadenas, verificando la integridad del léxico cromático y respetando la restricción de las unidades en estado lock.

MINI-CICLO 1: Intercambiar ruedas (swap)

Estado: Completo

Objetivo: Permitir que dos ruedas intercambien su posición en la máquina.

Checklist:

- Crear el método swap(int wheel1, int wheel2) en la clase SlotMachine.

- Implementar la lógica para validar los índices ingresados (recordando la regla de límites: si es menor a 1 se usa 1, si es mayor al máximo se usa el tamaño máximo y de igual forma la validación de posiciones posibles a ingresar).
- Intercambiar los objetos Wheel dentro de la estructura de datos (ArrayList).
- Actualizar visualmente la posición de ambas ruedas en el lienzo (Canvas) usando el método actualizarRuedas() que construimos en el ciclo anterior.

MINI-CICLO 2: Fijar y soltar ruedas (lock / unlock)

Estado: Completo

Objetivo: Permitir que el usuario bloquee una rueda para que no gire, y que pueda desbloquearla después.

Checklist:

- Agregar un atributo booleano de estado (ej. isLocked) en la clase Wheel.
- Implementar un identificador visual sutil en la clase Wheel para cuando esté bloqueada (ej. cambiar el color del borde o dibujar un pequeño candado/cuadro).
- Crear los métodos lock(int wheel) y unlock(int wheel) en SlotMachine.

Modificar la lógica de todos los métodos de giro (spin) para que ignoren (no giren) las ruedas que se encuentren en estado bloqueado.

MINI-CICLO 3: Rotar una rueda por pasos (spin con steps)

Estado: Completo

Objetivo: Girar una rueda específica una cantidad determinada de posiciones en el catálogo.

Checklist:

- Crear el método spin(int wheel, int steps) en SlotMachine.
- Desarrollar la lógica matemática para avanzar "n" pasos dentro de nuestro catálogo de símbolos (maneja el desbordamiento si llega al final de la lista de símbolos).
- **Requisito crítico de usabilidad:** Implementar un bucle que muestre la animación del movimiento paso a paso (usando el wait del Canvas) exclusivamente si el simulador está en modo visible. // IA implementativa

MINI-CICLO 4: Configuración forzada (spin con setSymbols)

Estado: Completo

Objetivo: Dejar la máquina en una configuración exacta proporcionada por el usuario.

Checklist:

- Crear el método spin(String[] setSymbols) en SlotMachine.

- Validar que las posiciones del arreglo coincidan con las ruedas existentes.
- Forzar a cada rueda respectiva a actualizar su símbolo visual según el texto que llega en el arreglo.
- Validar si la nueva configuración forzada es ganadora llamando al método `isJackpot()`.

MINI-CICLO 5: Pruebas de unidad y validación

Estado: Completo

Objetivo: Cumplir con los estándares de calidad y entrega del proyecto.

Partes que lo componen (Checklist):

- Crear la clase de pruebas `SlotMachineC2Test` evaluando los modos invisibles (qué debería hacer y qué no debería hacer el sistema).
- Programar los 2 casos de prueba de aceptación listos para la sustentación.
- Crear los 2 casos de prueba colectivos (`SlotMachineCC2Test`) siguiendo la convención de nombres con las iniciales (`accordingDeOmShould...`).

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

Finalizamos la totalidad de los objetivos propuestos para esta etapa. Se logró completar cada mini-ciclo del Ciclo 2, integrando funciones para la manipulación de componentes como el intercambio y bloqueo de piezas, además de las variaciones del giro. La robustez del sistema se validó con una suite de pruebas que confirma la gestión de premios y la protección ante entradas inválidas.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Se dedicaron cerca de 10 a 12 horas por persona. Este esfuerzo se concentró en robustecer los métodos contra fallos, sincronizar el estado visual de las piezas inmovilizadas y estructurar pruebas en JUnit que verifican las reglas de negocio de forma independiente a la interfaz gráfica.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El hito principal fue consolidar operaciones complejas como lock y la configuración por arreglos, manteniendo un encapsulamiento riguroso. Destaca la capacidad de proteger elementos bloqueados frente a cambios globales de

configuración, validado mediante la prueba
accordingDcOmShouldNotAlterLockedWheelsWhenForcingConfiguration.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

La dificultad técnica residió en la automatización de pruebas que se veían interrumpidas por elementos visuales y retardos de animación. **Solución:** Se aplicó el criterio de visibilidad condicional, asegurando que en el entorno de pruebas (@Before) la máquina operara en modo invisible. Se refactorizó la lógica para validar el estado mediante el método ok(), independizando la verificación lógica de la renderización en el Canvas.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Resaltamos la adopción efectiva de la programación dirigida por pruebas, analizando exhaustivamente los comportamientos esperados y prohibidos. El compromiso a futuro es profundizar en la desvinculación de componentes y perfeccionar el rendimiento gráfico para evitar parpadeos visuales durante la ejecución.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

El Desarrollo Basado en Pruebas (TDD) resultó fundamental. Al contar con la suite SlotMachineC2Test, pudimos realizar ajustes en el código con la certeza de no comprometer funcionalidades previas, optimizando los tiempos de verificación.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Escuela Colombiana de Ingeniería. (2026). *Desarrollo Orientado Por Objetos [DOPO-POOB] Proyecto Inicial Ciclo No 2/4. Documento de requisitos y diagrama UML.* (Fuente esencial para definir la arquitectura y contratos de los métodos).
- Escuela Colombiana de Ingeniería. (2026). *DOPO - Instructivo de Pruebas Unitarias y Colectivas.* (Guía para la nomenclatura técnica accordingDcOm...).

- JUnit Team. (2025). *JUnit 4 Documentation*. Documentación técnica para la implementación de pruebas.

